PERSON FINDER · A COMPLETE WALKTHROUGH

# Finding a person from a single sentence

How a text-and-image embedding model was measured, fine-tuned and turned into a live video tool that boxes the person you describe, explained step by step in plain words.

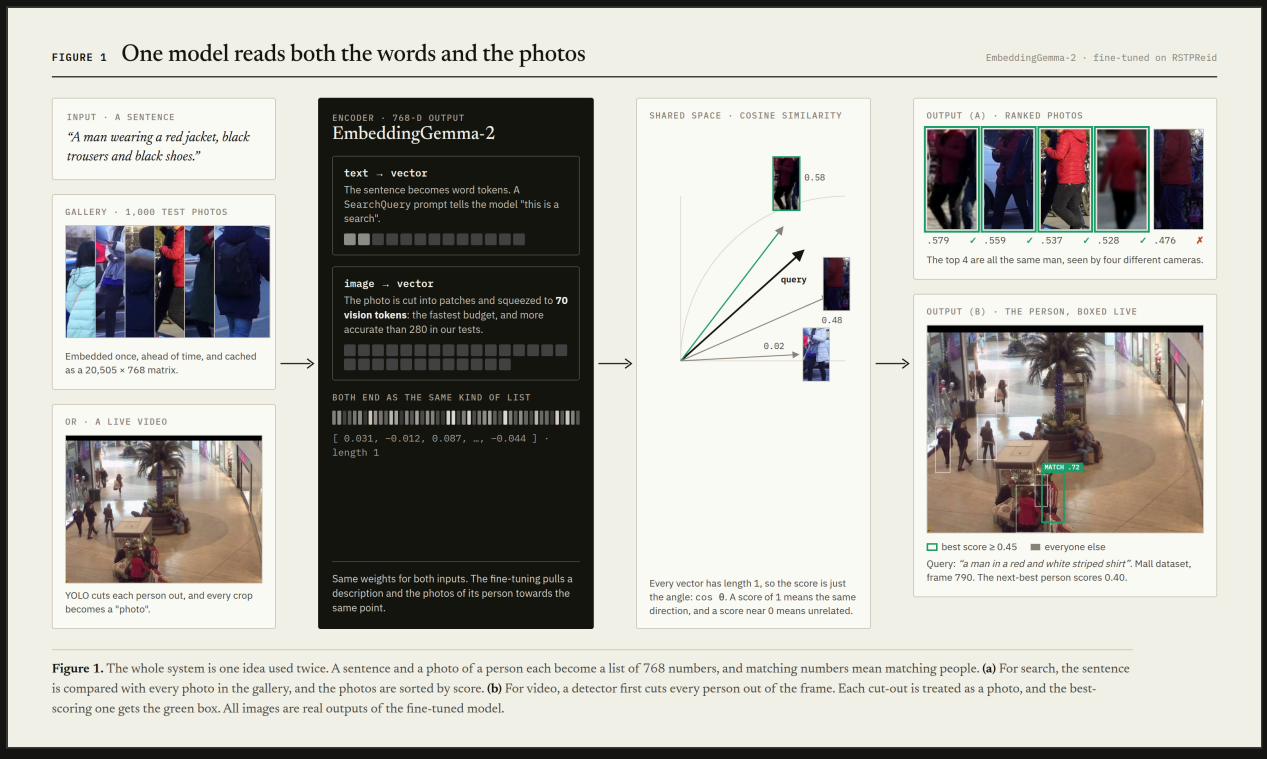



**Figure 1.** The whole system is one idea used twice. A sentence and a photo of a person each become a list of 768 numbers, and matching numbers mean matching people. **(a)** For search, the sentence is compared with every photo in the gallery, and the photos are sorted by score. **(b)** For video, a detector first cuts every person out of the frame. Each cut-out is treated as a photo, and the best-scoring one gets the green box. All images are real outputs of the fine-tuned model.

**40 → 71%**
first answer correct, before and after fine-tuning

**≈ 26 fps**
live video, end to end, on one GPU

**0.72 vs 0.40**
the right shopper against the next-best, in a mall crowd

BEFORE YOU START

# How to read this document

Every step answers three questions: **what** was done, **when** in the project it happens, and **why** it was done that way and not some other way. Wherever a technical word first appears, a box called *"In simple words"* explains it without jargon.

> **IN SIMPLE WORDS**
> The project teaches a computer to do what a security guard does when a parent says, "I lost my son, he's wearing a red jacket and black trousers". The computer looks at every person in the photos or the video and points at the one who fits best.


01 The problem
02 The data
03 The model and embeddings
04 Step 1 · Measure before changing anything
05 How the grades work
06 Step 2 · Fine-tuning
07 Step 3 · The results
08 Step 4 · The image-search website
09 Step 5 · Live video
10 Step 6 · A real crowd: the Mall dataset
11 Limits and next steps
12 Glossary
13 Run it yourself

01 · THE PROBLEM

# Searching for people with words

Cameras record thousands of people an hour. When someone is lost or needs to be found, nobody has a photo of them to search with, only a description.

This task is called **text-based person retrieval**. The input is a sentence such as *“a woman with long black hair in a white coat and blue jeans”*, and the output is a ranked list of person photos with the best match first. A second version of the same task works on video: the input is a sentence and a live camera feed, and the output is a box around the right person in every frame.

### WHY WORDS AND NOT A PHOTO?

Searching with a photo of the person (re-identification) only works when you already have a photo. A witness or a parent only has memory and words.

### WHY IS IT HARD?

The same red coat looks dark red in the shade and almost black at night. Descriptions are short and vague, and many people wear similar clothes.

> **IN SIMPLE WORDS · RETRIEVAL**
> “Retrieval” means fetching the right items from a big pile. A search engine retrieves web pages, and this project retrieves people.

# RSTPReid: 4,101 people, 41,010 sentences

The project uses the public **RSTPReid** dataset: 20,505 photos of 4,101 different people taken by 15 street cameras. Each photo comes with two English descriptions written by two different people.

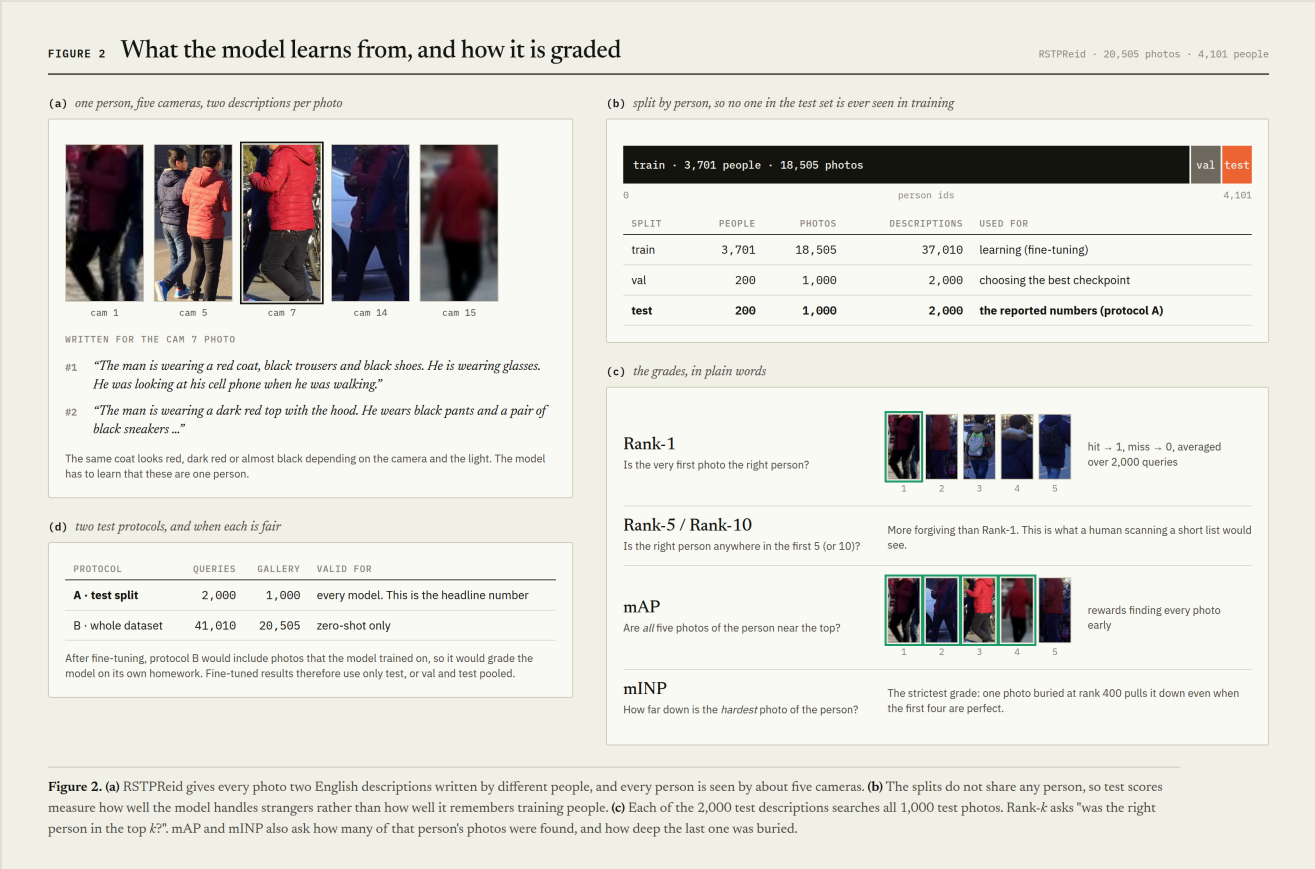



*Figure 2. One person seen by five cameras, how the dataset is split, and what each grade measures.*

## The split, and why it is done by person

| SPLIT | PEOPLE | PHOTOS | USED FOR |
|---|---|---|---|
| train | 3,701 | 18,505 | teaching the model |
| val | 200 | 1,000 | picking the best saved version during training |
| test | 200 | 1,000 | the final, reported grade. It is touched only once |

**Why split by person?** If the same person appeared in both training and testing, the model could memorise their face and clothes, and the test would look easier than real life. Because nobody in the test set was ever seen during training, the test measures the real skill: finding *strangers*.

> **IN SIMPLE WORDS · TRAIN / VAL / TEST**
> Think of a school year. The **train** set is the homework the student practises on. The **val** set is the mock exam that tells the teacher which week the student was at their best. The **test** set is the final exam, opened once at the end.

03 · THE MODEL · *WHEN:* CHOSEN AT THE START

# EmbeddingGemma-2, and what an “embedding” is

The model is Google's **EmbeddingGemma-2**. It does one thing: it turns a piece of text, *or* an image, into a list of 768 numbers. The same model handles both kinds of input.

> **IN SIMPLE WORDS · EMBEDDING**
> An embedding is like GPS coordinates for meaning. A sentence about a red jacket and a photo of a red jacket get coordinates that are close together. A sentence about a white coat gets coordinates far away. Finding the right person then becomes “find the nearest point on the map”.

## How closeness is measured: cosine similarity

Every list of 768 numbers is scaled to length 1, which is called *L2-normalised*. Two such lists can then be compared by the angle between them. The **cosine** of that angle is 1 when they point the same way, 0 when they are unrelated, and negative when they point apart. Ranking a gallery means computing one cosine per photo and sorting.

## Three settings that matter, and why

| SETTING | VALUE | WHY |
|---|---|---|
| Number format | `bfloat16` | Uses half the memory of normal 32-bit numbers, so the model runs fast on the GPU. The other 16-bit format, float16, is not supported by this model. |
| Text prompt | `SearchQuery` | The model was trained to treat text differently depending on the job. This prompt tells it “this sentence is a search”, which ranks better than plain text. |
| Vision tokens | `70` | An image is cut into small pieces called “tokens”. The model allows 70 to 1,120 tokens. Person crops are small and blurry, so 70 was both the fastest and the most accurate in the sweep (280 was the worst). |

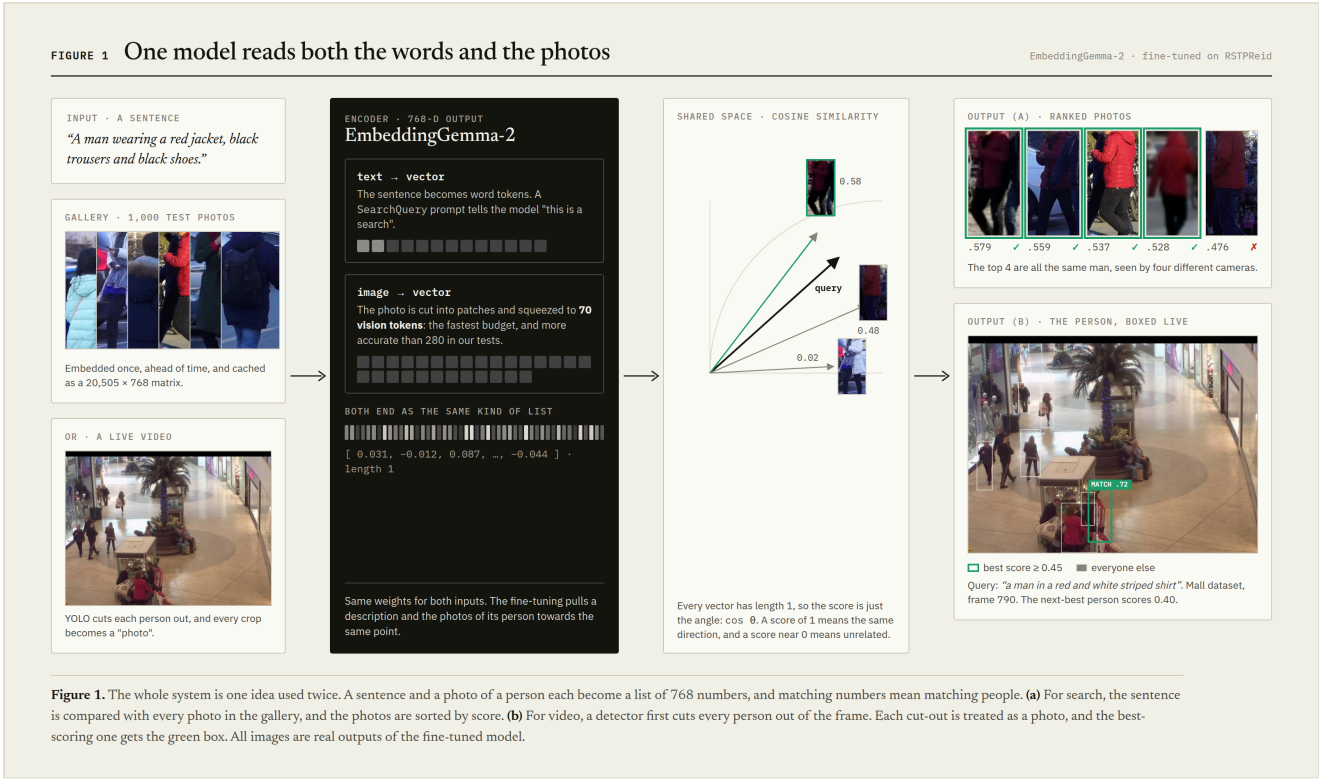



**Figure 1.** The whole system is one idea used twice. A sentence and a photo of a person each become a list of 768 numbers, and matching numbers mean matching people. **(a)** For search, the sentence is compared with every photo in the gallery, and the photos are sorted by score. **(b)** For video, a detector first cuts every person out of the frame. Each cut-out is treated as a photo, and the best-scoring one gets the green box. All images are real outputs of the fine-tuned model.

*Figure 1. One model reads both words and photos into the same space. Search and video are two uses of the same idea.*

04 · STEP 1 · *WHEN:* FIRST, BEFORE ANY TRAINING · `TBPR_EX.IPYNB`

# Measure the model as it is: the zero-shot baseline

You cannot know if training helped unless you know where you started. So the first experiment uses the model exactly as Google released it.

> **IN SIMPLE WORDS · ZERO-SHOT**
> "Zero-shot" means the model is tested on a task without any practice on it, like asking someone to referee a sport they have only read about.

1. **Encode all 20,505 photos once** and save them to `outputs/image_embeddings.npy`. This takes about 15 minutes. *Why save them?* Photos don't change, so there is no reason to recompute them for every search. The website reuses this file.
2. **Encode all 41,010 descriptions** with the `SearchQuery` prompt.
3. **Rank**: for every description, compute the cosine with every photo on the GPU, in chunks so that memory never overflows, and record where the right person landed.

| ZERO-SHOT, TEST SPLIT | RANK-1 | RANK-10 | MAP |
|---|---|---|---|
| 280 vision tokens | 35.5 | 70.0 | 25.0 |
| 70 vision tokens | **40.1** | **75.4** | **28.8** |

**What this taught us.** Out of the box, the model already finds the right person first 40% of the time among 1,000 photos, which is a real but modest skill. Searching the *whole* dataset (20,505 photos) drops Rank-1 to 10%. Re-running with random galleries of growing size showed why: Rank-1 falls steadily from 46% at 250 photos to 10% at 20,505. The task gets harder with more people to confuse, not because some photos are worse.

> **WHY THIS ORDER**
> Measuring first fixes a fair reference point, with the same split, the same metrics and the same code. Every later number is compared to it.

05 · THE GRADES

# Rank-1, Rank-5, mAP, mINP, without the maths

| GRADE | QUESTION IT ANSWERS | PLAIN EXAMPLE |
|---|---|---|
| **Rank-1** | Is the first photo the right person? | The guard points once. Right or wrong? |
| **Rank-5 / Rank-10** | Is the right person among the first 5 / 10? | The guard shows a short list, and the parent spots their child. |
| **mAP** | Are *all* photos of that person near the top? | Not just one sighting: every camera that saw them. |
| **mINP** | How deep is the *hardest* photo of that person? | The one blurry, dark sighting. Is it still found? |

The test runs each of the 2,000 test descriptions against the 1,000 test photos (**protocol A**). Every grade is an average over those 2,000 searches. All grading code lives in one shared file, `tbpr.py`, so the zero-shot and fine-tuned notebooks are graded in exactly the same way.

> **WHY SEVERAL GRADES?**
> One number hides things. A model can be good at the first guess (Rank-1) but leave the person's other photos buried (low mAP). Reporting all of them shows both.

06 · STEP 2 · *WHEN:* AFTER THE BASELINE · `TBPR_FINETUNE.IPYNB`

# Fine-tuning: teach the model what matters for people

The original model learned from general images and text on the internet. It has never been told that “dark red hooded top” and a blurry street crop of a red coat are the same person. Fine-tuning shows it 37,010 examples of exactly that.

> **IN SIMPLE WORDS · FINE-TUNING**
> Fine-tuning is extra practice on one specific job. The model keeps everything it already knows, and its numbers are nudged a little at a time until it gets better at this job.

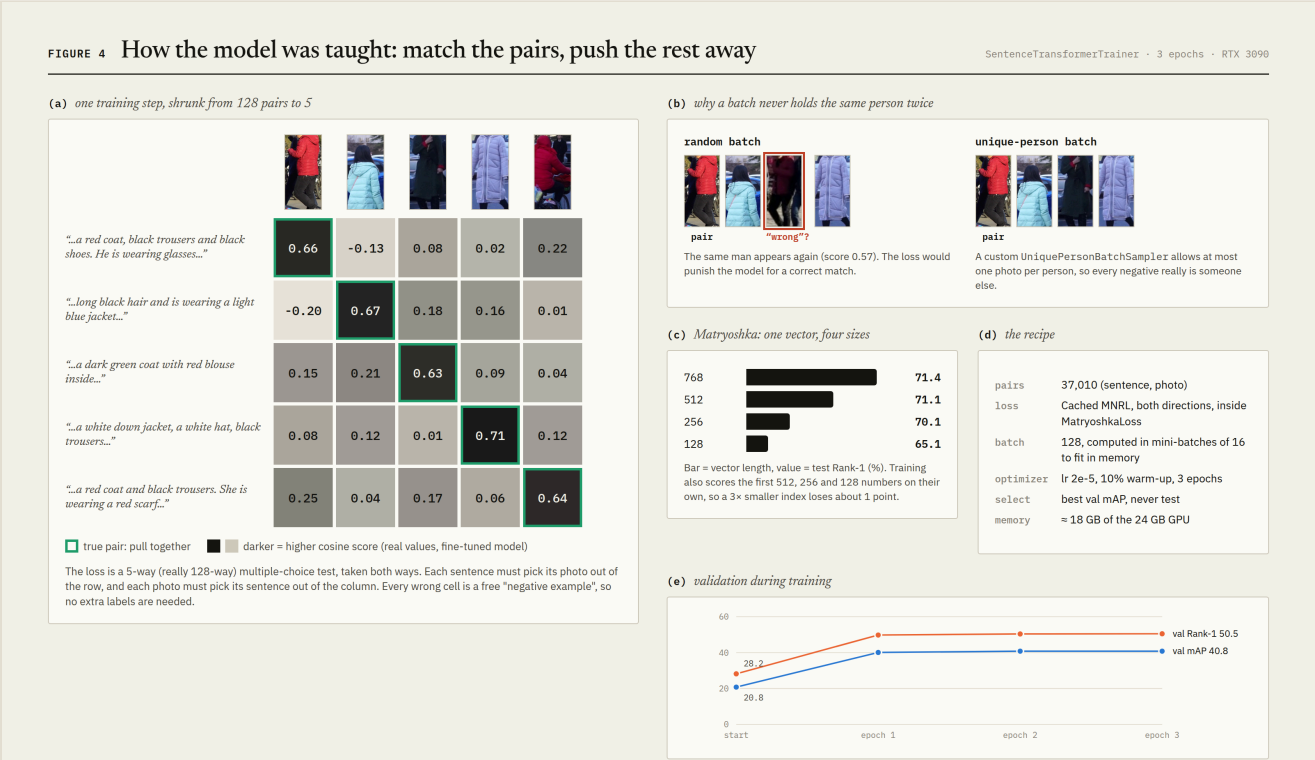



*Figure 4. One training step (a), the batch rule (b), nested vector sizes (c), the recipe (d) and the validation curve (e).*

## The loss: a multiple-choice test, taken both ways

Each step takes 128 (sentence, photo) pairs. For each sentence, the correct photo is the one it was written for, and the other 127 photos are wrong answers. The model is rewarded for giving the right pair the highest score in its row. The same test is then run the other way: each photo must pick its sentence. This is **contrastive learning**, using `CachedMultipleNegativesRankingLoss`.

- **Why 128 pairs?** More wrong answers per question make a harder, more useful test.

- **Why “cached”?** 128 images at once do not fit in 24 GB of GPU memory. The cached version computes them 16 at a time and combines the results, which gives the same maths with less memory.
- **Why both directions?** Search goes text to image, but training image to text as well makes the shared space more balanced.

## The batch rule: never two photos of the same person

Each person has about five photos. In a random batch of 128, the same person sometimes appears twice, and the loss would then call a *correct* match “wrong”. A small custom sampler, `UniquePersonBatchSampler`, builds every batch from 128 *different* people. **Why it matters:** without it the model would be punished for being right, which slowly teaches it the wrong thing.

## Matryoshka: one vector, four sizes

The loss is also applied to the first 512, 256 and 128 numbers of each vector on their own (`MatryoshkaLoss`). The model learns to put the most useful information first, like Russian nesting dolls. **Why:** a 256-number index is 3× smaller and faster, and after fine-tuning it loses only about one Rank-1 point.

## Settings, and the reason for each

| SETTING | VALUE | WHY |
|---|---|---|
| learning rate | 2e-5 | Small steps, so the model improves at this task without forgetting what it already knows. |
| warm-up | first 10% of steps | Starts even gentler, because the first updates are the noisiest. |
| epochs | 3 (≈ 33 min each) | The validation grade stops improving after epoch 1, so more epochs would only waste time. |
| model selection | best val mAP | The test set is never used to choose, so the final grade stays honest. |
| GPU memory | ≈ 18 GB | It fits on one RTX 3090 (24 GB). |

07 · STEP 3 · *WHEN:* AFTER TRAINING, ONCE

# The results

FIGURE 6 What fine-tuning bought, and where the limits are

protocol A · 2,000 queries × 1,000 photos · 200 unseen people

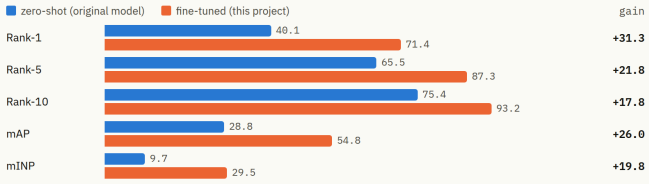
**71.4%** — from 40.1% — Rank-1: the first photo is the right person

**93.2%** — from 75.4% — Rank-10: the right person is on the first screen

**54.8** — from 28.8 — mAP: all of a person's photos found early

**3.1×** — 9.7 → 29.5 mINP — the hardest photo is found much higher

(a) *zero-shot vs fine-tuned, same 70-token setting*




Pooling val and test (2,000 photos and 400 people) makes the task harder, but the ranking holds: Rank-1 goes from 26.7 to 55.9.

(b) *fewer image tokens was better, and faster*

| ZERO-SHOT, VISION TOKENS | RANK-1 | MAP |
|---|---|---|
| 280 | 35.4 | 25.0 |
| **70** (used everywhere) | **40.1** | **28.8** |

RSTPReid crops are small and blurry, so extra tokens add detail that does not exist. At 70 tokens, each crop also needs 4× fewer image tokens, which helps keep live video fast.

(c) *a crowd the model never saw: the Mall dataset*

| DETECTOR INPUT | PRECISION | RECALL | COUNT ERROR |
|---|---|---|---|
| 640 px · conf 0.40 (demo) | 97.7 | 39.0 | 18.7 |
| 1280 px · conf 0.25 | 92.4 | **66.3** | **8.8** |

Against 62,315 hand-labelled heads in 2,000 frames. The language side transfers, because each test description scored its shopper 0.60–0.72. The limit is the detector: far-away and partly hidden people get no box, so they cannot be chosen.

**Figure 6.** All numbers are from the notebooks in this repository (`outputs/finetuned/results_test.csv`, `outputs/results_main.csv`, `outputs/mall/results_detection.csv`). Fine-tuning on 3,701 people nearly doubles every retrieval grade on 200 people it has never seen. The Mall experiment shows that, in a dense crowd, the next gain will come from the detector rather than from the language model.

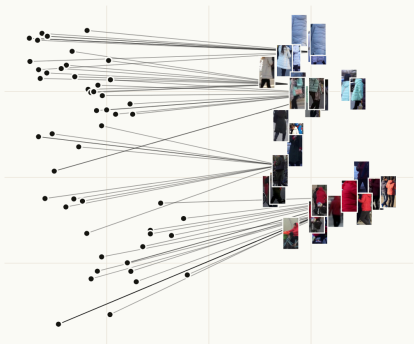
*Figure 6. Every grade on the 200 unseen test people, before and after fine-tuning.*

Fine-tuning nearly doubles every grade. **678** test searches moved up to first place, and 51 dropped out of it. The right person is now in the top 10 for 93% of searches.

FIGURE 3 Fine-tuning pulls each description onto its own person

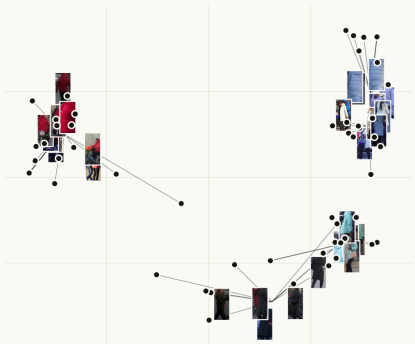
real embeddings · 6 test people · 30 photos · 60 descriptions

(a) *before: the original model, zero-shot*



**0.73** own person · **0.66** other people · **80%** sentences whose top photo (of 30) is right

(b) *after: fine-tuned on 3,701 other people*



**0.51** own person · **0.12** other people · **100%** sentences whose top photo (of 30) is right

(c) *how to read it*

a photo, placed by its 768 numbers

a description

line to the centre of *its* person

768 numbers cannot be drawn, so they are squashed to 2 with PCA, which keeps the two directions where the points differ most. Short lines mean a description already lands next to the right person.

(d) *the gap that does the work*

Average cosine score of a description with photos of its own person and with photos of other people. Only the distance between the two matters.

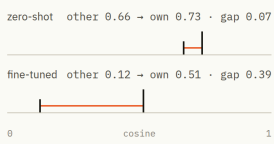



**Figure 3.** Each panel shows real model outputs for six test-set people that the model never trained on. **(a)** Out of the box, photos group with photos and sentences with sentences (the well-known "modality gap"), and every sentence is about equally close to everyone. **(b)** After contrastive fine-tuning, the descriptions land on top of the photos they describe. People dressed alike, such as the two in red, share a neighbourhood, but within it each sentence still puts its own person first: all 60 sentences here. **(d)** The gap between "own" and "other" grows from 0.07 to 0.39. That gap is what lets one threshold, 0.45, separate the right person from the crowd in video.

*Figure 3. Real embeddings of six unseen people, squashed to 2-D. Fine-tuning moves each sentence onto its person.*

**Why it works, in one picture:** before training, a sentence was about equally close to everyone (0.73 to its own person, 0.66 to others). After training, the gap is wide (0.51 against 0.12). That wide gap is what lets a single threshold separate the right person from the crowd in video later.




*Searches the original model got wrong at first place and the fine-tuned model gets right. Aqua frame: correct person.*

08 · STEP 4 · *WHEN:* AFTER THE MODEL IS FINAL · `DEMO/SERVER.PY`

# The image-search website

A small web server, written with Python's built-in HTTP library and no web framework, loads the fine-tuned model and the pre-computed photo embeddings. When you type a sentence, it encodes only that sentence (a few milliseconds), scores it against all photos with one matrix multiplication on the GPU, and returns the top 5.

- **Why pre-compute the photos?** Encoding 20,505 photos takes minutes, while encoding one sentence takes milliseconds. Doing the slow part once makes every search instant.
- **Why must the file match the model?** Embeddings from two different models live on two different "maps". Mixing them would compare GPS coordinates from two different planets.
- **Why only known image names are served:** the server never reads a file path the browser invents, so nobody can ask it for files outside the dataset.

```
GET  /api/example          a random test description and its person id
POST /api/search           {query, gallery: "test" | "all"}  →  top-5 photos + scores
GET  /imgs/<name>          only names that exist in the dataset
```

09 · STEP 5 · *WHEN:* AFTER SEARCH WORKED · DEMO/TRACK_PERSON.PY

# Live video: from a frame to one green box

The model only understands photos of single people. Video is turned into exactly that: a stream of single-person photos.

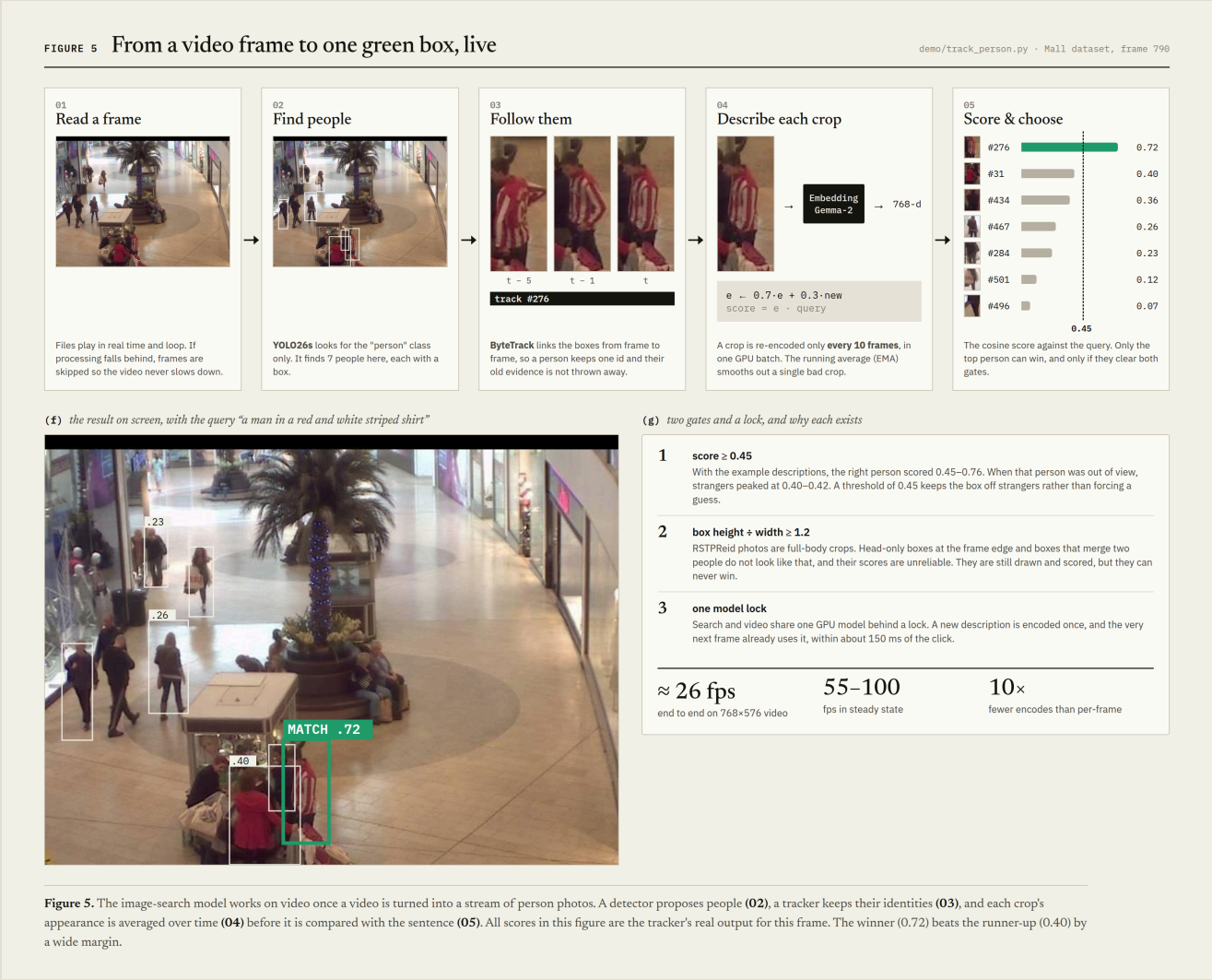



**Figure 5.** The image-search model works on video once a video is turned into a stream of person photos. A detector proposes people **(02)**, a tracker keeps their identities **(03)**, and each crop's appearance is averaged over time **(04)** before it is compared with the sentence **(05)**. All scores in this figure are the tracker's real output for this frame. The winner (0.72) beats the runner-up (0.40) by a wide margin.

*Figure 5. The five stages, with the tracker's real output for one frame of the Mall dataset.*

| STAGE | WHAT | WHY THIS CHOICE |
|---|---|---|
| Detect | YOLO26s, “person” class only | Small and fast, and it finds where people are so that the language model only looks at people. |
| Track | ByteTrack gives each person an id | A person keeps their id across frames, so evidence collected over time is not thrown away. |
| Re-encode | each track every 10 frames, in one GPU batch | The person rarely changes in a third of a second. This saves about 90% of the work. |
| Smooth | EMA: `e ← 0.7·e + 0.3·new` | One blurry crop cannot flip the decision, because the track keeps a running memory. |
| Gate 1 | score ≥ 0.45 | The right person scored 0.45–0.76, while strangers peaked at 0.40–0.42 when that person was absent. Below 0.45, no one gets the box. |
| Gate 2 | box height ÷ width ≥ 1.2 | Training photos are full bodies. Head-only boxes and boxes covering two people score unreliably, so they can never win. |

> **IN SIMPLE WORDS · EMA**
> An exponential moving average is a memory with fading weight. Every new look counts for 30%, and everything seen before counts for 70%. Recent looks matter most, but no single look can take over.

**Speed.** About 26 frames per second end to end on 768×576 video, and 55–100 fps once every track is encoded. In the website, a background thread plays the video in real time and skips frames if it falls behind, so the video never slows down. A new sentence takes effect on the very next frame, about 150 ms after the click.

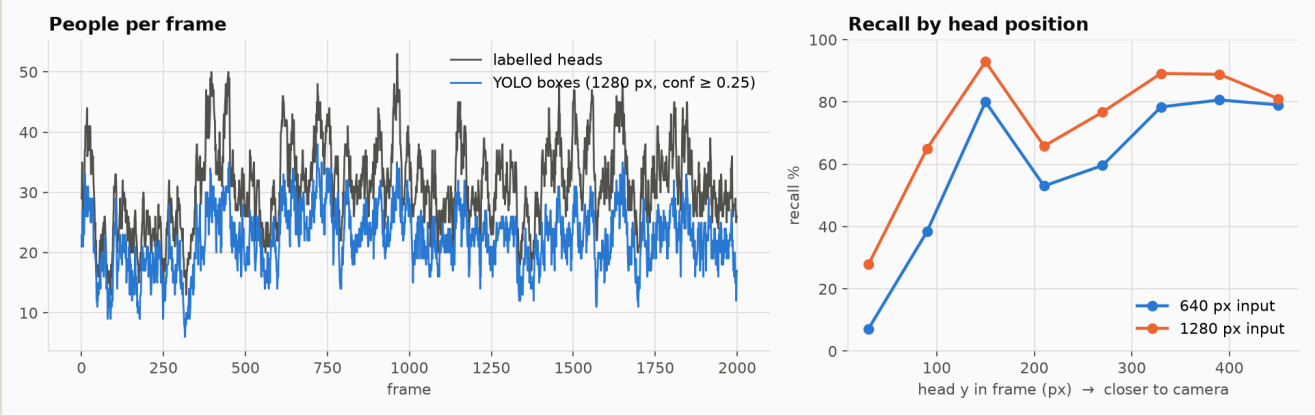
# A real crowd: the Mall dataset

Everything so far was tested on clean, cut-out photos. A shopping mall is the real thing: 13–53 people in each frame, people hiding behind each other, and tiny figures far from the camera.

The Mall dataset (Chen et al., BMVC 2012) has 2,000 frames from one camera. A human put a dot on **every head**, 62,315 dots in total. There are no names or ids, so it can grade one question exactly: *does the detector see the people?* The frames were also turned into a video, `videos/mall.mp4`, which plays in the website.

## How the detector is graded

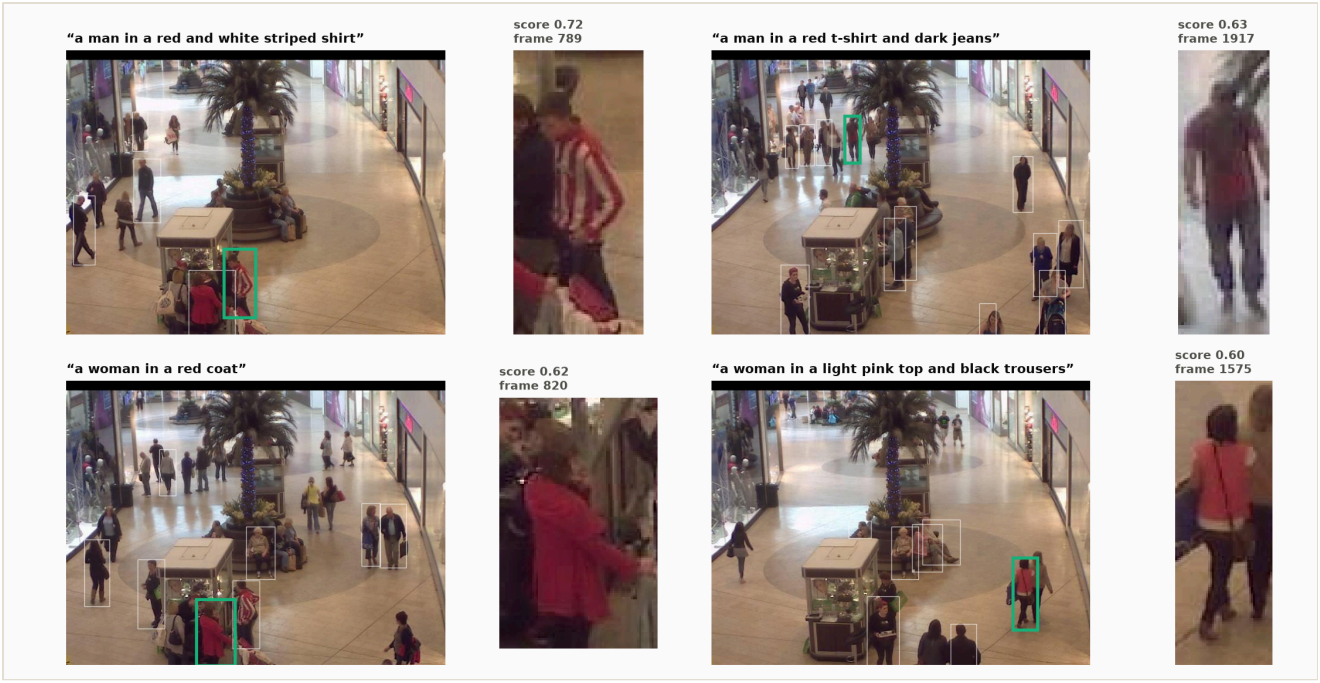
Each head dot may be matched to at most one person box, and only if the dot sits in the top 45% of that box. The optimal one-to-one pairing is found with the Hungarian algorithm. Matched dots count as hits, unmatched dots are misses, and unmatched boxes are false alarms.




*Left: people per frame, labelled against detected. Right: the share of people found at each height in the frame. People near the top of the image are far away and tiny.*

| DETECTOR SETTING | PRECISION | RECALL | COUNT ERROR / FRAME |
|---|---|---|---|
| 640 px, confidence 0.40 (the demo's default) | 97.7% | 39.0% | 18.7 |
| 1280 px, confidence 0.25 | 92.4% | **66.3%** | **8.8** |

> **IN SIMPLE WORDS · PRECISION AND RECALL**
> **Precision** asks: when the detector says “person”, how often is it right? **Recall** asks: of all the real people, how many did it find? The demo's detector is almost never wrong, but in a crowd it misses many far-away people.




*Four descriptions of shoppers, the frame where each scored highest, and the boxed crop. All four are the right person.*

**What this taught us.** The language model transfers to a place it never saw: each description finds its shopper among 20–30 people, with best scores of 0.60–0.72. The weak link in a crowd is the detector, because a person who never gets a box can never be chosen. Doubling the detector's input size to 1280 px finds 70% more people.

# What it cannot do yet, and what would come next

- **Small and hidden people.** In dense crowds, about a third of people get no box even at 1280 px. The next step would be a detector trained on crowds, or tiling the frame into overlapping pieces.
- **Similar clothes.** Two people in red share a neighbourhood in the embedding space (Figure 3b). Details such as “glasses” or “scarf” help, but a vague sentence stays ambiguous.
- **Low frame rates.** The Mall camera records under 2 frames per second, so ByteTrack often breaks tracks and the EMA memory restarts. Normal 25–30 fps cameras do not have this problem.
- **Language.** Training used English descriptions only.
- **Responsible use.** This is a research demo on public research datasets. Searching for real people in real footage needs consent, legal grounds and human review of every match.

# Every term, in one line

| | |
|---|---|
| **Embedding** | A list of numbers (here 768) that captures what a text or image means, so similar things get similar lists. |
| **Cosine similarity** | How much two lists point the same way: 1 is the same, 0 is unrelated. |
| **L2-normalised** | Scaled to length 1, so only the direction counts. |
| **Zero-shot** | Used on a task with no practice on it. |
| **Fine-tuning** | Extra training on one specific task, starting from a trained model. |
| **Contrastive loss** | Training by “match the right pair, push away the wrong ones”. |
| **Batch** | The group of examples the model studies in one step (here 128). |
| **Negative** | A wrong answer in the batch that the model should score low. |
| **Epoch** | One full pass over the training data. |
| **Learning rate** | How big each training step is. |
| **Matryoshka** | Training so the first part of a vector works on its own. |
| **Vision tokens** | The small pieces an image is cut into before the model reads it. |
| **bfloat16** | A 16-bit number format: half the memory, nearly the same accuracy. |
| **Rank-k** | Share of searches with the right person in the top k. |
| **mAP / mINP** | How early *all* / the *hardest* of a person's photos are found. |
| **Detector (YOLO)** | Finds where people are in a frame and draws boxes. |
| **Tracker (ByteTrack)** | Gives each box a stable id from frame to frame. |
| **EMA** | A running average that favours recent values. |
| **Precision / recall** | How often a detection is right / how many real people are found. |
| **Hungarian algorithm** | Finds the best one-to-one pairing between two sets, here boxes and head dots. |

# Commands

```
# 1. measure zero-shot, fine-tune, then stress-test on the mall (each notebook saves its
outputs)
.venv/bin/jupyter execute --inplace TBPR_ex.ipynb
.venv/bin/jupyter execute --inplace TBPR_finetune.ipynb     # ≈ 18 GB GPU, ≈ 1.7 h
.venv/bin/jupyter execute --inplace TBPR_mall.ipynb

# 2. the website: image search on /, live video on /video
.venv/bin/python demo/server.py --port 8000 \
    --model models/embeddinggemma2-rstpreid --embeddings
outputs/finetuned/image_embeddings.npy \
    --video videos/*.mp4 --video-query "a man in a red and white striped shirt"

# 3. video in a desktop window (type a new sentence in the terminal + Enter)
.venv/bin/python demo/track_person.py --model models/embeddinggemma2-rstpreid \
    --source videos/mall.mp4 --query "a woman in a red coat"

# 4. rebuild the figures in docs/diagrams and this PDF
.venv/bin/python docs/diagrams/render.py
.venv/bin/python docs/explainer/build_pdf.py
```